Application development

Building during development

- Buildroot is mainly a *final integration* tool: it is aimed at downloading and building **fixed** versions of software components, in a reproducible way.
- When doing active development of a software component, you need to be able to quickly change the code, build it, and deploy it on the target.
- The package build directory is temporary, and removed on `make clean`, so making changes here is not practical
- Buildroot does not automatically "update" your source code when the package is fetched from a version control system.
- Three solutions:
  - Build your software component outside of Buildroot during development. Doable for software components that are easy to build.
  - Use the `local SITE_METHOD` for your package
  - Use the `<pkg>_OVERRIDE_SRCDIR` mechanism

Building code for Buildroot

- The Buildroot cross-compiler is installed in `$(HOST_DIR)/bin`
- It is already set up to:
  - generate code for the configured architecture
  - look for libraries and headers in `$(STAGING_DIR)`
- Other useful tools that may be built by Buildroot are installed in `$(HOST_DIR)/bin`:
  - `pkg-config`, to find libraries. Beware that it is configured to return results for *target* libraries: it should only be used when cross-compiling.
  - `qmake`, when building Qt applications with this build system.
  - `autoconf`, `automake`, `libtool`, to use versions independent from the host system.
- Adding `$(HOST_DIR)/bin` to your `PATH` when cross-compiling is the easiest solution.

Building code for Buildroot: C program

Building a C program for the host

```
$ gcc -o foobar foobar.c
$ file foobar
foobar: ELF 64-bit LSB executable, x86-64, version 1...
```

Building a C program for the target

```
$ export PATH=$(pwd)/output/host/bin:\$PATH
$ arm-linux-gcc -o foobar foobar.c
$ file foobar
foobar: ELF 32-bit LSB executable, ARM, EABI5 version 1...
```

Building code for Buildroot: pkg-config

Using the system `pkg-config`

```
$ pkg-config --cflags libpng
-I/usr/include/libpng12

$ pkg-config --libs libpng
-lpng12
```

Using the Buildroot `pkg-config`

```
$ export PATH=$(pwd)/output/host/bin:\$PATH

$ pkg-config --cflags libpng
-I.../output/host/arm-buildroot-linux-uclibcgnueabi/sysroot/usr/include/libpng16

$ pkg-config --libs libpng
-L.../output/host/arm-buildroot-linux-uclibcgnueabi/sysroot/usr/lib -lpng16
```

Building code for Buildroot: autotools

- Building simple *autotools* components outside of Buildroot is easy:

```
$ export PATH=../buildroot/output/host/bin/:\$PATH
$ ./configure --host=arm-linux
```

- Passing `–host=arm-linux` tells the configure script to use the cross-compilation tools prefixed by `arm-linux-`.
- In more complex cases, some additional `CFLAGS` or `LDFLAGS` might be needed in the environment.

Building code for Buildroot: CMake

- Buildroot generates a *CMake toolchain file*, installed in `output/host/share/buildroot/toolchainfile.cmake`
- Tells *CMake* which cross-compilation tools to use
- Passed using the `CMAKE_TOOLCHAIN_FILE` *CMake* option
- `https://cmake.org/cmake/help/latest/manual/cmake-toolchains.7.html`
- With this file, building *CMake* projects outside of Buildroot is easy:

```
$ cmake -DCMAKE_TOOLCHAIN_FILE=.../buildroot/output/host/share/buildroot/toolchainfile.cmake .
$ make
$ file app app: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked...
```

Building code for Buildroot: Meson

- Buildroot generates a Meson *cross file*, installed in `output/host/etc/meson/cross-compilation.conf`
- Tells *Meson* which cross-compilation tools to use
- Passed using the `–cross-file` *Meson* option
- `https://mesonbuild.com/Cross-compilation.html`
- With this file, building *Meson* projects outside of Buildroot is easy:

```
$ mkdir build
$ meson --cross-file=.../buildroot/output/host/etc/meson/cross-compilation.conf ..
$ ninja
$ file app app: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked...
```

Building code for Buildroot: `environment-setup`

- Enable `BR2_PACKAGE_HOST_ENVIRONMENT_SETUP`
- Installs an helper shell script `output/host/environment-setup` that can be sourced in the shell to define a number of useful environment variables and aliases.
- Defines: `CC`, `LD`, `AR`, `AS`, `CFLAGS`, `LDFLAGS`, `ARCH`, etc.
- Defines `configure` as an alias to run a *configure* script with the right arguments, `cmake` as an alias to run *cmake* with the right arguments
- Drawback: once sourced, the shell environment is really only suitable for cross-compiling with Buildroot.

Building code for Buildroot: environment-setup

```
$ source output/host/environment-setup
 _           _ _     _                 _
| |__  _   _(_) | __| |_ __ ___   ___ | |_
| '_ \| | | | | |/ _` | '__/ _ \ / _ \| __|
| |_) | |_| | | | (_| | | | (_) | (_) | |_
|_.__/ \__,_|_|_|\__,_|_|  \___/ \___/ \__|

       Making embedded Linux easy!

Some tips:
* PATH now contains the SDK utilities
* Standard autotools variables (CC, LD, CFLAGS) are exported
* Kernel compilation variables (ARCH, CROSS_COMPILE, KERNELDIR) are exported
* To configure do "./configure \$CONFIGURE_FLAGS" or use
  the "configure" alias
* To build CMake-based projects, use the "cmake" alias
```

```
$ echo \$CC
/home/thomas/projets/buildroot/output/host/bin/arm-linux-gcc
$ echo \$CFLAGS
-D_LARGEFILE_SOURCE -D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -Os -D_FORTIFY_SOURCE=1
$ echo \$CROSS_COMPILE
/home/thomas/projets/buildroot/output/host/bin/arm-linux-
$ alias configure alias configure='./configure --target=arm-buildroot-linux-gnueabihf --host=arm-buildroot-linux-gnueabihf
    --build=x86_64-pc-linux-gnu --prefix=/usr --exec-prefix=/usr --sysconfdir=/etc --localstatedir=/var
    --program-prefix='
```

`local` site method

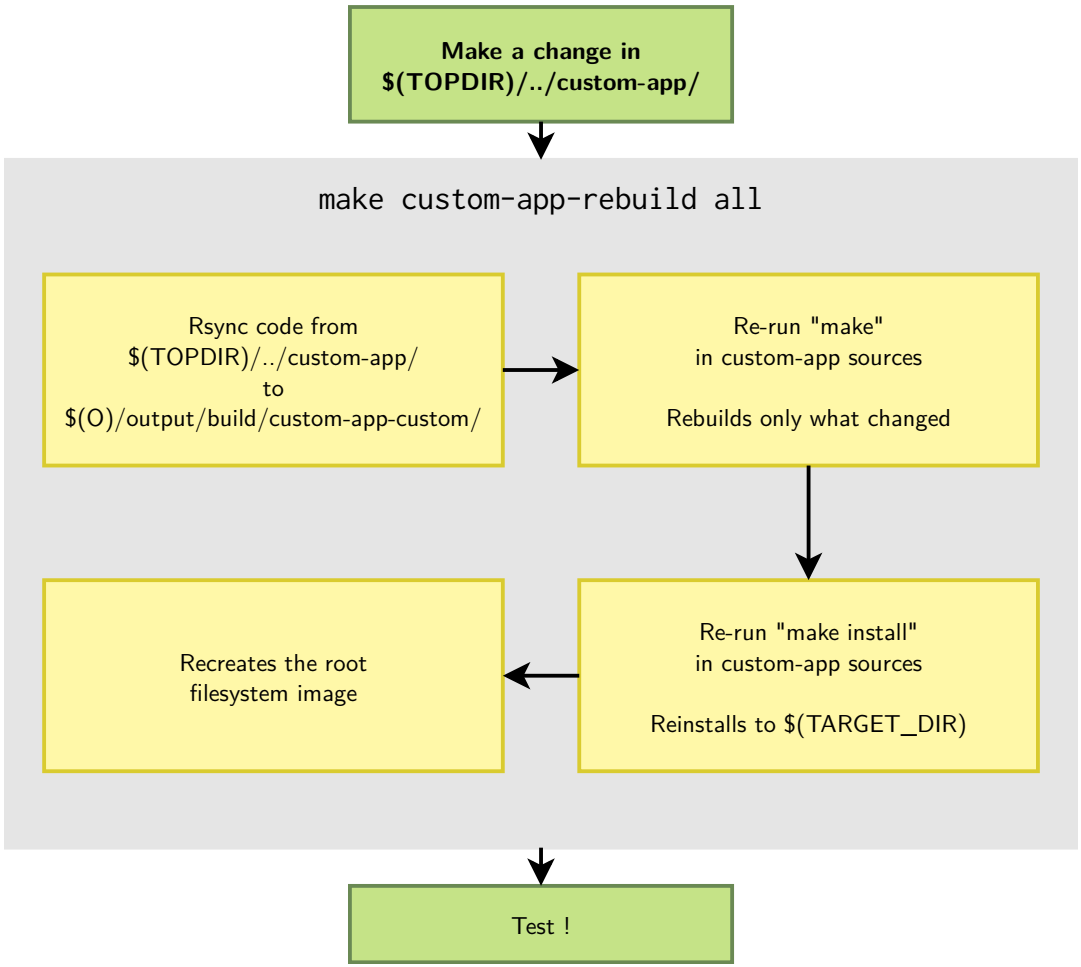
- Allows to tell Buildroot that the source code for a package is already available locally
- Allows to keep your source code under version control, separately, and have Buildroot always build your latest changes.
- Typical project organization:
  - `buildroot/`, the Buildroot source code
  - `external/`, your `BR2_EXTERNAL` tree
  - `custom-app/`, your custom application code
  - `custom-lib/`, your custom library
- In your package `.mk` file, use:

```
<pkg>_SITE = $(TOPDIR)/../custom-app
<pkg>_SITE_METHOD = local
```

Effect of `local` site method

- For the first build, the source code of your package is *rsync'ed* from `<pkg>_SITE` to the build directory, and built there.
- After making changes to the source code, you can run:
  - `make <pkg>-reconfigure`
  - `make <pkg>-rebuild`
  - `make <pkg>-reinstall`
- Buildroot will first *rsync* again the package source code (copying only the modified files) and restart the build from the requested step.

`local` site method workflow



<pkg>_OVERRIDE_SRCDIR

- The `local` site method solution is appropriate when the package uses this method for all developers
  - Requires that all developers fetch locally the source code for all custom applications and libraries
- An alternate solution is that packages for custom applications and libraries fetch their source code from version control systems
  - Using the `git`, `svn`, `cvs`, etc. fetching methods
- Then, locally, a user can **override** how the package is fetched using `<pkg>_OVERRIDE_SRCDIR`
  - It tells Buildroot to not *download* the package source code, but to copy it from a local directory.
- The package then behaves as if it was using the `local` site method.

Passing <pkg>_OVERRIDE_SRCDIR

- <pkg>_OVERRIDE_SRCDIR values are specified in a *package override file*, configured in BR2_PACKAGE_OVERRIDE_FILE, by default $(CONFIG_DIR)/local.mk.

Example local.mk

```
LIBPNG_OVERRIDE_SRCDIR = $(HOME)/projects/libpng
LINUX_OVERRIDE_SRCDIR = $(HOME)/projects/linux
```

Debugging: debugging symbols and stripping

- To use debuggers, you need the programs and libraries to be built with debugging symbols.
- The `BR2_ENABLE_DEBUG` option controls whether programs and libraries are built with debugging symbols
  - Disabled by default.
  - Sub-options allow to control the amount of debugging symbols (i.e. gcc options `-g1`, `-g2` and `-g3`).
- The `BR2_STRIP_strip` option allows to disable or enable stripping of binaries on the target.
  - Enabled by default.

Debugging: debugging symbols and stripping

- With `BR2_ENABLE_DEBUG=y` and `BR2_STRIP_strip=y`
  - get debugging symbols in `$(STAGING_DIR)` for libraries, and in the build directories for everything.
  - stripped binaries in `$(TARGET_DIR)`
  - Appropriate for **remote debugging**
- With `BR2_ENABLE_DEBUG=y` and `BR2_STRIP_strip` disabled
  - debugging symbols in both `$(STAGING_DIR)` and `$(TARGET_DIR)`
  - appropriate for **on-target debugging**

Debugging: remote debugging requirements

- To do remote debugging, you need:
  - A **cross-debugger**
    - With the *internal toolchain backend*, can be built using `BR2_PACKAGE_HOST_GDB=y`.
    - With the *external toolchain backend*, is either provided pre-built by the toolchain, or can be built using `BR2_PACKAGE_HOST_GDB=y`.
  - **gdbserver**
    - With the *internal toolchain backend*, can be built using `BR2_PACKAGE_GDB=y` + `BR2_PACKAGE_GDB_SERVER=y`
    - With the *external toolchain backend*, if `gdbserver` is provided by the toolchain it can be copied to the target using `BR2_TOOLCHAIN_EXTERNAL_GDB_SERVER_COPY=y` or otherwise built from source like with the internal toolchain backend.

Debugging: remote debugging setup

- On the target, start *gdbserver*
  - Use a TCP socket, network connectivity needed
  - The *multi* mode is quite convenient
  - `$ gdbserver –multi localhost:2345`
- On the host, start `<tuple>-gdb`
  - `$ ./output/host/bin/<tuple>-gdb <program>`
  - `<program>` is the path to the program to debug, with debugging symbols
- Inside *gdb*, you need to:
  - Connect to the target: `(gdb) target extended-remote <ip>:2345`
  - Tell the target which program to run: `(gdb) set remote exec-file myapp`
  - Set the path to the *sysroot* so that *gdb* can find debugging symbols for libraries: `(gdb) set sysroot ./output/staging/`
  - Start the program: `(gdb) run`

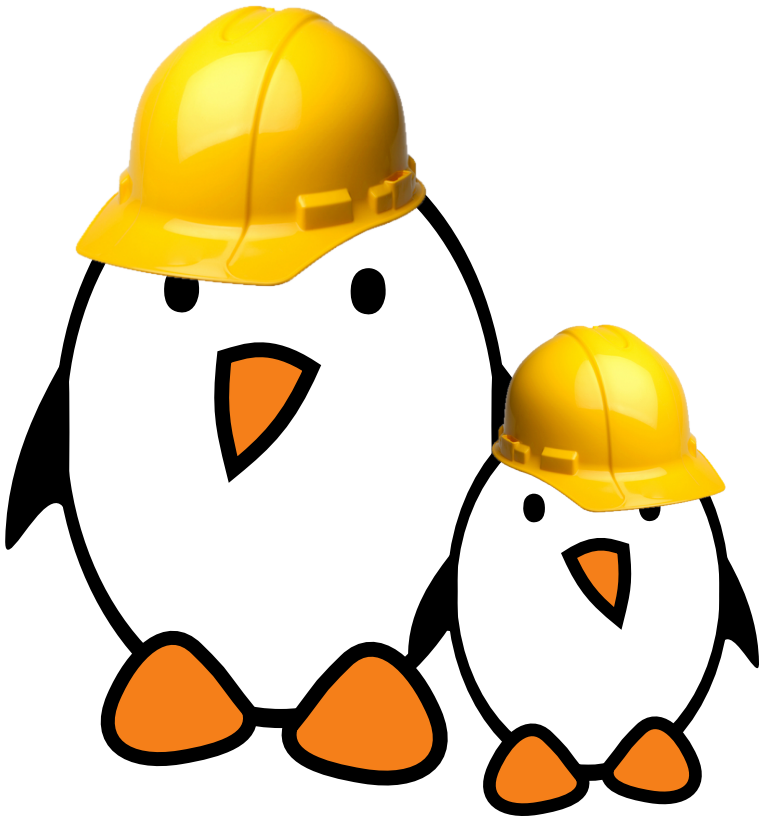
Debugging tools available in Buildroot

- Buildroot also includes a huge amount of other debugging or profiling related tools.
- To list just a few:
  - strace
  - ltrace
  - LTTng
  - perf
  - sysdig
  - sysprof
  - OProfile
  - valgrind
- Look in `Target packages` → `Debugging, profiling and benchmark` for more.

Generating a SDK for application developers

- If you would like application developers to build applications for a Buildroot generated system, without building Buildroot, you can generate a SDK.
- To achieve this:
  - Run `make sdk`, which prepares the SDK to be relocatable
  - Tarball the contents of the *host* directory, i.e. `output/host`
  - Share the tarball with your application developers
  - They must uncompress it, and run the `relocate-sdk.sh` script
- **Warning**: the SDK must remain in sync with the root filesystem running on the target, otherwise applications built with the SDK may not run properly.

Practical lab - Application development



- Build and run your own application
- Remote debug your application
- Use `<pkg>_OVERRIDE_SRCDIR`